

RESEARCH ARTICLE

ST-D3QN: Advancing UAV Path Planning With an Enhanced Deep Reinforcement Learning Framework in Ultra-Low Altitudes

ANPING YANG¹, JING HUAN¹, QI WANG¹, (Member, IEEE), HUALONG YU¹, (Member, IEEE), AND SHANG GAO, (Member, IEEE)

School of Computer Science and Engineering, Jiangsu University of Science and Technology, Zhenjiang 212000, China

Corresponding authors: Jing Huan (jhuan@just.edu.cn) and Qi Wang (wangqi@just.edu.cn)

ABSTRACT Unmanned aerial vehicles (UAVs) have increasingly become indispensable instruments for surveillance, reconnaissance, and various missions due to their advanced capabilities. In intricate and dynamic operational contexts, especially during ultra-low altitude missions, inadequate communication and coordination impede UAVs' capacity to avoid other UAVs and moving objects. In this instance, attaining accurate path planning for UAVs constitutes a significant technological problem that requires immediate attention. This paper introduces an innovative algorithm, ST-D3QN, which leverages deep reinforcement learning (DRL) to optimize UAV trajectories based on optimizing the Dueling Double Deep Q-Network (D3QN) and A* (A-star) algorithm. Our approach refines the optimization of sparse rewards by toggling between the two methods, thereby maximizing the long-term benefits for UAVs. The greedy strategy has been refined to prevent the algorithm from becoming ensnared in local optima. Our proposed method is benchmarked against state-of-the-art path planning algorithms through meticulously designed simulation tests. The experimental results indicate that the ST-D3QN algorithm achieves the fastest convergence rate during training and excels in optimizing sparse rewards. It also significantly reduces the risk of collisions and enhances the success rates of UAV path planning, establishing itself as an efficient and effective solution for determining the shortest path for UAVs in complex environments.

INDEX TERMS UAV, ultra-low altitude path planning, D3QN, sub-targets, sparse rewards.

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs) are autonomous aircraft [1] that operate without human intervention, relying on embedded computer systems or remote control. Over the past two decades, UAVs have attracted considerable attention from industry and academia [2], emerging as a key research domain. Their exceptional maneuverability, autonomy, and flexibility have led to a broad spectrum of real-world applications [3]. The expansion of UAV applications across commercial, search and rescue, and military sectors has driven researchers to enhance their agility and maneuverability. Path planning [4] is a critical component of UAV autonomous mission planning, aiding UAVs in

navigating toward targets while circumventing obstacles by adhering to a predefined global path. It depends on the environmental information, represented as maps, to identify obstacle-free zones for pathfinding.

Many types of UAVs are used simultaneously in outdoor tasks. However, inefficient interconnection and communication infrastructures have made UAV path planning in ultra-low altitude and complex conditions more critical [5]. Ultra-low-altitude UAVs face several static barriers and mobility risks like other UAVs and erratically flying flocks of birds, which make navigation dangerous. In complex scenarios, UAVs' autonomous obstacle avoidance is insufficient, raising stability and dependability concerns. On the other hand, although deep reinforcement learning (DRL) techniques [6] have provided novel prospects for UAV path optimization, they, too, face significant challenges in

The associate editor coordinating the review of this manuscript and approving it for publication was Guillermo Valencia-Palomo¹.

practical applications. For instance, the learning process of DRL algorithms is initially sluggish, and the insufficient acquisition of environmental information can lead the training process to converge prematurely on local optima. Moreover, the issue of sparse reward signals severely affects the adaptability and convergence efficiency of these algorithms in complex dynamic environments [7]. These challenges limit the widespread application of DRL in UAV flight control and diminish the assurance of efficient task execution by UAVs operating under intricate environmental conditions. Consequently, it is imperative to design a novel control strategy that leverages the advantages of DRL to address the critical issues in path planning for UAVs in ultra-low altitude environments. Such an approach aims to overcome existing limitations and enhance the autonomous decision-making capabilities of UAVs in complex and dynamic settings.

This work introduces an algorithm to address the complexities of path planning in dynamic environments and to avoid obstacles for UAVs [8]. By assimilating environmental information, the algorithm enables UAVs to plan collision-free trajectories toward their destinations. The primary contributions of this paper are as follows:

- 1) We propose a novel algorithm, ST-D3QN, that optimizes UAV paths based on enhancing D3QN (Dueling Double Deep Q-Network) and A* (A-star) algorithms to address path optimization and dynamic obstacle avoidance for UAVs operating in ultra-low altitude environments. The algorithm focuses on reducing UAV collisions and task execution times. By minimizing the flight time of UAVs during task execution, significant performance improvements can be achieved for UAV path planning in dynamic environments.
- 2) We innovatively enhance traditional greedy strategies to tackle the slow learning speed in the initial stages and the inadequate acquisition of environmental information associated with DRL algorithms. The refined greedy strategy dynamically adjusts based on the UAV's learning progress, facilitating rapid learning and preventing entrapment in local optima during training.
- 3) To address the common issue of sparse rewards in DRL algorithms, we introduce a method where the sub-target sequences generated by the A* algorithm undergo fixed-value sampling. For each sub-target, constraint regions are defined to encourage UAV proximity to the sub-targets, guiding UAV training and optimizing sparse rewards.

The subsequent sections of this paper are organized as follows: Section II reviews the advancements in UAV path planning research, with a particular focus on the A* algorithm and D3QN algorithm utilized in this study. Section III delivers the system model and problem statement. Section IV presents the ST-D3QN approach developed in this research. Section V compares the ST-D3QN algorithm with other typical algorithms. Section VI summarizes the study's findings and suggests potential directions for future research.

II. RELATED WORKS

Path planning is crucial for UAV operations and has garnered global scientific interest. Critical concerns in UAV path planning include optimizing trajectories, ensuring integrity, determining optimal paths, enhancing flight efficiency, and guaranteeing resilience [4]. UAVs encounter challenges in path determination, prompting the creation and use of various path planning algorithms [9] to enhance energy efficiency and path optimization. Optimizing path planning protects UAVs from colliding with obstacles to their objective. Consequently, effective UAV flight path planning is essential for rapid and secure task execution, particularly in dynamic environments.

Many approaches have been introduced to address UAV path planning challenges, including Rapidly-exploring Random Tree (RRT), Artificial Potential Field (APF), A*, reinforcement learning (RL), and DRL [10], [11], [12], [13]. Huang et al. [14] proposed an energy-aware RRT method to identify feasible paths before optimization. Guo et al. [15] introduced TIME-BASED RRT* (HPO-RRT*), which employs lazy planning and cost-effective optimization to navigate hazards, though its performance is sensitive to sampling efficiency. Pan et al. [16] enhanced IAPF for multi-UAV coordination by mitigating local minima using a rotating potential field. Rao et al. [17] developed an APF-A* technique for quadcopters that produces a secure, rapid flight path, albeit requiring twin UAVs. Gao et al. [18] formulated a multi-objective planning model addressing trajectory minimization and positional inaccuracies. Furthermore, Bolorian et al. [19] introduced a GA- and A*-based 3D path planning method for bridge inspection to reduce flight time, and Zhang et al. [20] refined A* with bidirectional sector extension and variable steps to improve efficiency, cost, and safety. These methods, however, face limitations in complex, dynamic environments.

Advances in artificial intelligence have catalyzed the increasing application of DRL methods for path optimization. Several studies have focused on optimizing UAV path planning under various conditions. Ou et al. [21] introduced a Dueling Double Recursive Q-Network (D3RQN) that improves learning and execution in partly observable scenarios. However, its evaluation was limited to static settings. Similarly, Kong et al. [22] combined Deep Q-Network (DQN) techniques with an APF approach to enhance UAV routing in static environments. Peng et al. [23] proposed a Double Deep Q-Learning Network (DDQN) emphasizing freshness awareness. While Zhou et al. [24] employed a Soft Actor-Critic (SAC) method to improve convergence for static UAV planning. In contrast, Zhu et al. [25] developed an Asynchronous Advantage Actor-Critic (A3C) system for adaptive UAV routing on dynamic CSI maps. However, its reward learning mechanism does not address real-time collision avoidance. Yan et al. [26] presented a heuristic trajectory method that integrates ant colony optimization, chaotic mapping, and simulated annealing to overcome unknown obstacles in UAV-assisted MEC networks, albeit

under static offshore conditions. Another study by Yan et al. [27] applied an LSTM-based Deep Deterministic Policy Gradient (DDPG) for UAV task offloading in vehicular networks, prioritizing network efficiency over direct path design. Furthermore, Guan et al. [28] enhanced collision avoidance in unmanned surface vehicles using a hybrid A* and dynamic window technique, focusing on wet conditions rather than UAV applications. Kästner et al. [29] later combined a hybrid A* global planner with a DRL-based local planner for terrestrial robots in dynamic environments. Finally, Wang et al. [30] and Kim et al. [31] refined the D3QN approach for UAV navigation in static settings. In contrast, Wang et al. [32] extended it to dynamic environments with motion, task, and communication considerations. Overall, recent research chiefly targets improvements in D3QN to handle high-dimensional, dynamic inputs in UAV trajectory optimization.

These studies have concentrated on static or high-altitude dynamic scenarios using RRT, APF, A*, and DRL methods. However, these approaches assume static or limited information environments and struggle with inefficient sampling and optimal pathfinding in complex dynamic settings. In ultra-low altitude areas with dense obstacles, conventional methods fail to balance dynamic obstacle avoidance with achieving the shortest path, resulting in higher energy consumption and longer trajectories. Moreover, DRL methods are primarily evaluated in static scenarios where early training suffers from slow learning, limited environmental data, and susceptibility to local optima. In high-dimensional state spaces, sparse rewards further destabilize training, hindering timely UAV navigation. This paper addresses UAV path planning in complex ultra-low altitude environments requiring safety and path optimality. This research initiative seeks to provide more efficient and dependable solutions for UAV operations in ultra-low altitude environments.

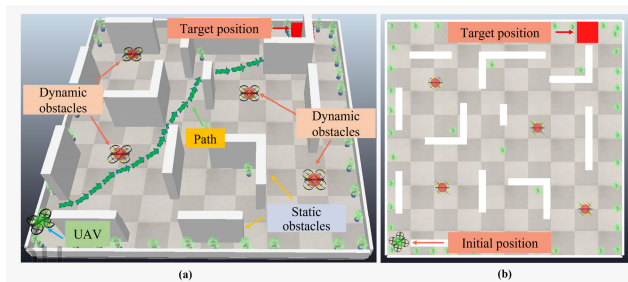


FIGURE 1. Description of ultra-low altitude environment (a) and UAV simulation environment (b) top-down view of the simulation environment.

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. PROBLEM STATEMENT

This study examines UAV path planning in highly blocked ultra-low altitude airspace [33], [34]. The objective is to develop an algorithm that enables UAVs to navigate autonomously, avoiding collisions and optimizing paths. Path

planning in ultra-low altitude airspace is challenging because of the dense presence of static and dynamic obstacles, including other UAVs. This study defines the ultra-low altitude environment as the area within a certain threshold, where UAVs and static obstacles maintain a consistent elevation [35]. In this airspace, UAVs are the only sentient entities, with adjacent UAVs functioning as dynamic barriers that adhere to their flight plans and lack predictive capabilities. UAVs may use onboard sensors to ascertain the real-time positions and velocities of adjacent UAVs and the locations of static impediments. The setting of Figure 1. UAVs are shown in green, and their detection range is illustrated in Figure 1(a). The orange UAV signifies various environmental types, whilst the red marking denotes the UAV's path planning objective. Figure 1(b) depicts the environment from an aerial perspective, with the orange arrow indicating the starting point of the UAV's path planning. UAVs may gather environmental data such as distances and velocities relative to adjacent UAVs, the target location, and fixed obstacles.

The environment scaling factor is defined as follows:

$$F = \frac{L_r}{L_s} \quad (1)$$

where L_r represents the actual size of the map, and L_s represents the size of the map in the simulation environment. In this paper, we set the value of F as 10.

The UAV seeks to maximize the cumulative reward of path planning by starting from the initial location and progressing toward the goal position at each time step while circumventing static and dynamic UAVs. The objective function is articulated as follows:

$$\arg \max_{\pi_{\theta}} E[t_g | S_{h,t}, \pi_{\theta}] \quad (2)$$

where the height h is treated as a constant, indicating fixed altitude flight. t_g represents the time the UAV takes to reach the target location from the initial position. π_{θ} denotes the strategy employed by the UAV to reach the target location successfully. $S_{h,t}$ signifies the sequence of state spaces available to the UAV in the environment.

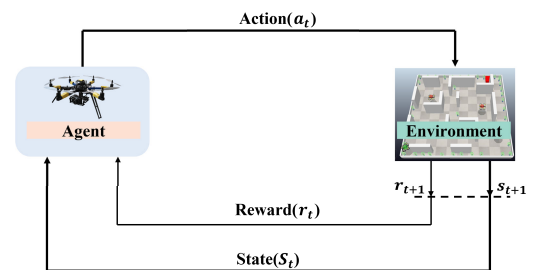


FIGURE 2. MDP for UAV operating in an environment.

B. MDP FOR PATH PLANNING

UAVs operate as agents within their environment. Hence, path planning may be conceptualized as a sequential

decision-making issue using Markov Decision Processes (MDP) [36], [37]. MDP comprises a tuple (S, A, R, P) . S denotes the set of possible states the agent may occupy inside the environment. A represents the agent's environmental engagement. R indicates the aggregate of all possible rewards the agent may get for each action executed under certain conditions. The state transition function P illustrates the changes in the dynamic environment resulting from an agent's actions. UAVs function like agents, engaging with their environment by executing tasks and receiving rewards in return. In Figure 2, the environment provides the UAV with the reward signal r_t after executing action a_t at time step t . The timestamp transitions from t to $t+1$, and the environment evolves from S_t to S_{t+1} . The UAV continually determines the subsequent action a_{t+1} using the new environmental condition S_{t+1} and reward r_t .

1) STATE SPACE

The environmental information perceived by UAVs mainly comprises three aspects: the state vector of the UAV, the vectors of target position and starting position, and the state vector of obstacles within the UAV's perceptual range. Consequently, the state space of the UAV is defined as:

$$S_{h,t} = \left[s_{u,t}, s_{p,t}, \sum_{i=0}^{N_o} s_{o,t}^i \right] \quad (3)$$

where $s_{u,t}$ denotes the state vector of the UAV, and $s_{p,t}$ represents the vector encompassing the initial and target positions. $\sum_{i=0}^{N_o} s_{o,t}^i$ signifies the state vectors of each obstacle within the perceptual range of the UAV when N_o obstacles are detected. The definition of $s_{u,t}$ is as follows:

$$s_{u,t} = [x_t, y_t] \quad (\forall t \in T) \quad (4)$$

where (x_t, y_t) represents the coordinate of the horizontal position of the unmanned aerial vehicle flying at a fixed altitude at time t . T denotes the set of time (all T mentioned in this paper represent the set of time). The definition of $s_{p,t}$ is as follows:

$$s_{p,t} = [x_0, y_0, x_g, y_g] \quad (\forall t \in T) \quad (5)$$

where (x_0, y_0) represents the coordinates of the initial position of the UAVs, and (x_g, y_g) denotes the positional coordinates of the UAV upon reaching the target location. The definition of $s_{o,t}^i$ is as follows:

$$s_{o,t}^i = [x_{it}, y_{it}] \quad (\forall t \in T) \quad (6)$$

where (x_{it}, y_{it}) represents the position and coordinates perceived by the unmanned aerial vehicle for the i -th obstacle.

2) ACTION SPACE

The effectiveness of actions in the action space depends on the physical constraints of the UAV [38]. To facilitate algorithm performance testing, the UAV's continuous actions were discretized into discrete actions [22]. Figure 3 illustrates the

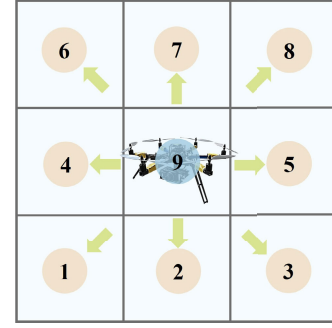


FIGURE 3. Action space.

discretized action space of the UAV, representing each specific action executed by the UAV. The continuous actions of the UAV were quantized into nine discrete actions, where the discrete action space is denoted as $A = [a_1, a_2, a_3, \dots, a_9]$, corresponding to the indices $[1, 2, 3, \dots, 9]$ in Figure 3. Hence, after executing an action, the position state of UAV changes. According to the expression in Equation (4), the current position coordinates of the UAV, denoted as (x_t, y_t) , are intended to transition to the following position coordinates, denoted as (x_{t+1}, y_{t+1}) . To achieve this transition, the UAV needs to take the action a_t , which is defined as follows:

$$a_t = (x_{t+1} - x_t, y_{t+1} - y_t) \quad (7)$$

In Equation (7), $(x_{t+1} - x_t)$ and $(y_{t+1} - y_t)$ are defined as follows:

$$\Delta x_t = (x_{t+1} - x_t) \quad (8)$$

$$\Delta y_t = (y_{t+1} - y_t) \quad (9)$$

However, in this environment, UAV can only execute one action from the action space at each timestamp t . Therefore, each action in the discretized action space can be defined as:

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} = \begin{bmatrix} (\Delta x_1, \Delta y_1) & (\Delta x_2, \Delta y_2) & (\Delta x_3, \Delta y_3) \\ (\Delta x_4, \Delta y_4) & (\Delta x_5, \Delta y_5) & (\Delta x_6, \Delta y_6) \\ (\Delta x_7, \Delta y_7) & (\Delta x_8, \Delta y_8) & (\Delta x_9, \Delta y_9) \end{bmatrix} \quad (10)$$

Due to the discretized action space, which dictates that the UAVs can only navigate path planning based on a grid map, thus, within the grid map, when the UAV is in a non-static state, its velocity scalar along the x and y axes, $|V_x| = |V_y| = 1$, meaning it can traverse only one grid cell per time step t . Therefore, the discretized action space of the UAV can be defined as:

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} = \begin{bmatrix} (-1, -1) & (0, -1) & (1, -1) \\ (-1, 0) & (1, 0) & (-1, 1) \\ (0, 1) & (1, 1) & (0, 0) \end{bmatrix} \quad (11)$$

The UAV selects a specific action to execute based on the action space outlined in Figure 3. Among these actions, action a_9 indicates that the UAV is stationary, denoted as $a_9 = (0, 0)$. In this state, while maintaining a constant flight altitude, there is no change in displacement along the x and y axes. When the UAV encounters multiple dynamic obstacles obstructing its path, choosing the stationary action is deemed safer than other actions. The selection of actions by the UAV is contingent upon environmental information, dynamically opting for the most optimal action.

3) STATE TRANSITION MODEL

The UAV's state transition adheres to the MDP framework. The UAV modifies the current state by interacting with the surroundings. The algorithm affects the transition probability distribution. The transition function $P(s_{t+1} | s_t, a)$ for the UAV described in this paper represents the probability of the UAV transitioning from its current state $s_t \in S$ to the next state $s_{t+1} \in S$ at time t after taking action a_t , where $a_t \in A$.

4) REWARD FUNCTION

In the considered environment, the reward function of autonomous UAVs can be defined as:

$$R = r_{\text{step}} + r_d + r_a + r_{\text{goal}} + r_{\text{sub-target}} \quad (12)$$

In Equation (12), the first item r_{step} represents the single-step reward of the UAV in the environment, which is associated with obstacles in the environment. However, the scalar velocity of the UAV is denoted by $|V|$, where $|V|$ represents the UAV's scalar velocity along the x or y -axis. When the UAV collides with obstacles in the environment, it incurs a penalty denoted by $-|V|F$. However, if the UAV navigates without collisions, it receives a reward denoted by $-|V|$. Therefore, r_{step} can be defined as follows:

$$r_{\text{step}} = \begin{cases} -|V|F & \text{if collision} \\ -|V| & \text{if no collision} \end{cases} \quad (13)$$

The second item, r_d , represents the distance reward function, which reflects the state of the UAV's distance from the intended target. The distance d between the target and the UAV's current position is calculated using the Euclidean distance calculation method. d can be defined as follows:

$$d = \text{Dis}(P_u, P_g) \quad (14)$$

where P_u represents the coordinates (x, y) of the current position of the UAV, and P_g represents the coordinates (x_g, y_g) of the target position. The Euclidean distance calculation formula $\text{Dis}(x, y)$ in this paper can be defined as follows:

$$\text{Dis}(x, y) = \sqrt{\sum_{i=1}^{N_d} (x_i - y_i)^2} \quad (15)$$

where N_d represents the dimension, and due to the constant flight altitude, the value of N_d is 2.

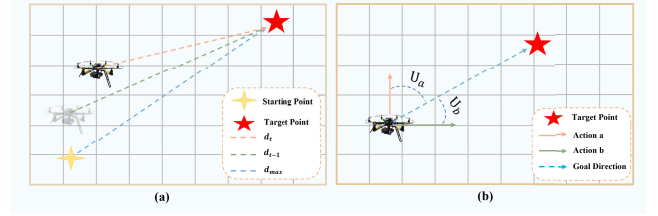


FIGURE 4. The geometric constraints of the reward function include (a) distance reward constraint, and (b) angular reward constraint.

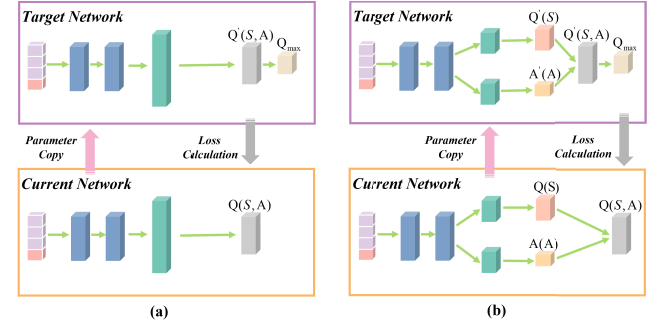


FIGURE 5. Neural network structures: (a) DQN network structure, (b) D3QN network structure.

The distance reward function aims to account for how much the UAV's position varies relative to the target position over time. At the current time t , the UAV's distance from the target position is d_t , and at the previous time $t-1$, the distance of the UAV from the target position is d_{t-1} . The distance change Δd can be defined as:

$$\Delta d = d_t - d_{t-1} \quad (16)$$

The depiction of the distance reward constraint is presented in Figure 4(a). Consequently, the reward function r_d can be defined as follows:

$$r_d = \begin{cases} -1 - \eta_1 \Delta d & \text{if } \Delta d < 0 \\ -\eta_2 e^{\frac{d_1}{\lambda_d}} & \text{if } \Delta d \geq 0 \end{cases} \quad (17)$$

where the parameters η_1 and η_2 represent the coherence coefficients of the distance reward function, with $\eta_1 = 0.03$ and $\eta_2 = 0.6$. Meanwhile, λ_d serves as the environmental maximization factor associated with the initial position of UAV and the target point. In the environment, if $\lambda_d = d_{\text{max}}$, then λ_d is a constant. Thus, d_{max} can be defined as:

$$d_{\text{max}} = \text{Dis}(P_{u0}, P_g) \quad (18)$$

where P_{u0} denotes the initial position coordinates of the UAV in the training environment, i.e., the coordinates (x_0, y_0) of the UAV's initial position. Thus, the distance reward r_d is dynamically provided as the UAV moves dynamically. As the UAV moves towards the target point, it receives rewards in the direction of distance. Conversely, when the UAV moves away from the target point, it incurs more significant penalties.

The third item, r_a , is the heading reward function, directing the UAV toward the exploration target. Figure 4(b) illustrates the heading reward constraint alongside the heading reward function r_a as defined by Equation (18). UAV incurs a -2 penalty for heading deviations exceeding 90 degrees. The UAV's reward increases as the heading deviance falls below 90 degrees, directing it toward the designated location. The heading coefficient of the UAV, η_3 , is used in the heading reward function of this research. Our study indicates that $\eta_3 = 0.01$, a more significant coefficient, accentuates the UAV's trajectory, perhaps confining it to local optima and hindering its objective attainment. In Equation (19), U represents the deviation angle of the UAV's heading.

$$r_a = \begin{cases} -\frac{\eta_3}{\cos U} & \text{if } |U| < 90^\circ \\ -2 & \text{if } |U| \geq 90^\circ \end{cases} \quad (19)$$

The fourth item, r_{goal} , represents the goal reward function. Upon reaching the target point, the UAV receives a maximal reward; however, if the UAV is located at any other position in the environment, no reward is obtained. Hence, the goal reward function r_{goal} can be defined as follows:

$$r_{\text{goal}} = \begin{cases} |V|F & \text{if goal} \\ 0 & \text{if no goal} \end{cases} \quad (20)$$

where the scalar speed of UAV is denoted by $|V|$, where $|V|$ presents the UAV's scalar velocity along the x or y -axis. F denotes the environmental scaling factor, the value of which is determined by equation (1). $|V|F$ represents the value corresponding to the maximum reward available in the environment.

The fifth item, $r_{\text{sub-target}}$, represents the sub-target reward constraint function. To optimize sparse rewards during the training of DRL algorithms. Sub-target reward limits restrict the UAV's local exploration range, hindering anonymous global searches and diminishing reward sparsity. The UAV will incur no environmental penalty for recognizing nearby sub-targets within its viewing range. The UAV will persistently incur environmental penalties when it fails to detect sub-targets. This incentive mechanism directs the UAV towards the appropriate sub-target orientation by designating the radiation area as $\mathfrak{R}$. UAVs may focus on local exploration under sub-target reward limitations, executing tasks while circumventing exploration challenges amidst little incentives. Consequently, the sub-target reward constraint function $r_{\text{sub-target}}$ is:

$$r_{\text{sub-target}} = \begin{cases} 0 & \text{if } \delta > \mathfrak{R} \\ -|V| & \text{if } \delta \leq \mathfrak{R} \end{cases} \quad (21)$$

where $\mathfrak{R}$ denotes the sensing signal range from the UAV to the target, defined as the detection radius of the sensor employed in this investigation. At the same time, δ signifies the distance between the sensor and the sub-target discovered in real time.

IV. PROPOSED ST-D3QN ALGORITHM

In this section, the fundamental principles of D3QN and A* algorithm are firstly introduced. Then, the proposed ST-D3QN path planning algorithm is elaborated upon in detail.

A. DUELING DOUBLE DEEP Q NETWORK (D3QN)

RL is an approach employing learning to find the optimal strategies by having an agent interact with the environment [39], based on the principles of Markov decision theory. Specially, Q-learning [40] is a classical method in the traditional RL methods. The definition of the expected benefit of the Q-value function is as follows:

$$Q(s_t, a_t) = Q(s_t, a_t) + \gamma \left[r(s_t, a_t) + \mu \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \quad (22)$$

where $s_t \in S_{h,t}$ and $s_{t+1} \in S_{h,t}$, $a_t \in A$ and $a_{t+1} \in A$, γ denotes the learning rate, and μ represents the discount factor. The fundamental task of most RL algorithms is to seek the optimal policy that maximizes the expected cumulative discounted reward [41]. Thus, the optimal policy π^* can be defined as:

$$\pi^*(s_t) = \operatorname{argmax}_{a_t} Q(s_t, a_t) \quad (23)$$

DRL [42] employs neural networks to approximate the value function of actions. This process effectively decreases the state space's dimensionality and promotes optimal policy learning. By calculating Equation (2) and Equation (22), the optimal policy π_θ^* of DRL can be defined as:

$$\pi_\theta^*(s_t) = \operatorname{argmax}_{a_t} Q(s_t, a_t, \theta) \quad (24)$$

The DQN algorithm [42] is one of the most classic DRL methods in the UAV path planning. To handle high-dimensional input-output, the DQN algorithm [43] employs neural networks to approximate the Q-value function. The Q-value function in DQN is defined as $Q_{DQN}(s_t, a_t, \theta)$ approximating $Q(s_t, a_t)$, where θ represents the neural network's training weights. Figure 5(a) illustrates the network architecture of DQN, comprising two structurally identical networks: The network with weight θ^- is the target network used to calculate the target Q-value, while the network with weight θ is the current network used to estimate the Q-value of the current state. DQN trains weights by minimizing the loss based on the discrepancy between the target and current state Q-values. Therefore, $Q_{DQN}(s_t, a_t, \theta)$ can be defined as:

$$\begin{aligned} Q_{DQN}(s_t, a_t, \theta) &= Q_{DQN}(s_t, a_t, \theta) + \gamma \left[r(s_t, a_t, \theta^-) \right. \\ &\quad \left. + \mu \max_{a_{t+1}} Q_{DQN}(s_{t+1}, a_{t+1}, \theta^-) - Q_{DQN}(s_t, a_t, \theta) \right] \end{aligned} \quad (25)$$

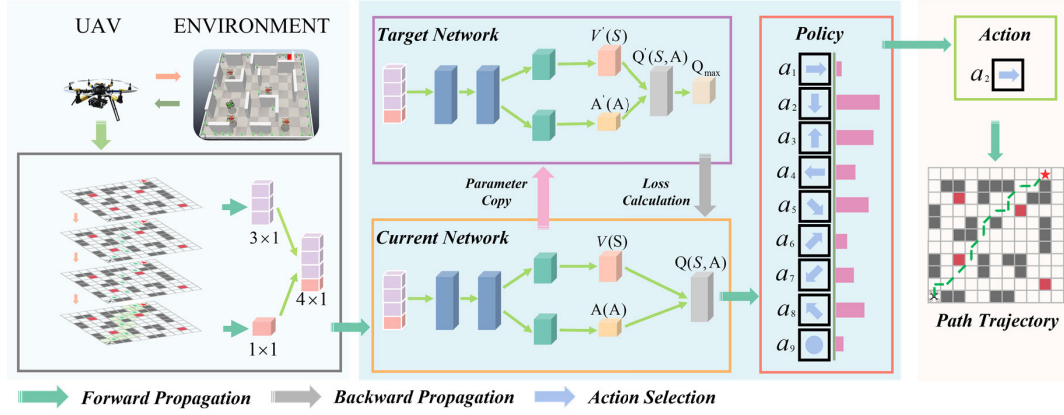


FIGURE 6. The algorithmic framework of ST-D3QN.

According to Equation (25), the target network can be defined as follows:

$$Q_{DQN}^{\text{target}}(s_t, a_t, \theta^-) = r(s_t, a_t, \theta^-) + \mu \max_{a_{t+1}} Q_{DQN}(s_{t+1}, a_{t+1}, \theta^-) \quad (26)$$

Therefore, equation (22) can be defined as:

$$Q_{DQN}(s_t, a_t, \theta) = Q_{DQN}(s_t, a_t, \theta) + \gamma [Q_{DQN}^{\text{target}}(s_t, a_t, \theta^-) - Q_{DQN}(s_t, a_t, \theta)] \quad (27)$$

The DQN algorithm updates the current network at each iteration, while the target network remains unchanged for a specific number of iterations. A predetermined number of iterations are employed to refresh the target network, and the training of the two networks occurs independently. Hence, the present network can undergo stable training. The alteration of the neural network parameters θ is determined by the loss function $L_{DQN}(\theta)$, which is defined as:

$$L_{DQN}(\theta) = E_{\pi_\theta} [(Q_{DQN}^{\text{target}}(s_t, a_t, \theta^-) - Q_{DQN}(s_t, a_t, \theta))^2] \quad (28)$$

We revise the learning parameters by calculating derivatives of the loss function. Stability in the optimization process is enhanced over time as the target network remains unchanged. Gradient updates can be formulated as follows:

$$\nabla_{\theta} L_{DQN}(\theta) = E_{\pi_\theta} [(Q_{DQN}^{\text{target}}(s_t, a_t, \theta^-) - Q_{DQN}(s_t, a_t, \theta)) \nabla_{\theta} Q_{DQN}(s_t, a_t, \theta)] \quad (29)$$

By utilizing Equation (25), the updating process of the weights θ for the current network in DQN can be defined as follows:

$$\theta_{t+1} = \theta_t + \gamma (Q_{DQN}^{\text{target}}(s_t, a_t, \theta^-) - Q_{DQN}(s_t, a_t, \theta_t)) \nabla_{\theta} Q_{DQN}(s_t, a_t, \theta_t) \quad (30)$$

where θ_t denotes the weight of the current network at time step t . Therefore, the updating process of the target network in DQN can be represented as follows:

$$\theta^- \leftarrow \theta \quad (31)$$

For adopting the UAV trajectory planning, the D3QN possesses a distinct network architecture at the end of the convolutional layers. Figure 5(b) illustrates the network structure of D3QN. Following several fully connected layers, D3QN divides the network into two branches: the state-value function $V(S)$ and the advantage function $A(S)$. The state-value function $V(S)$ evaluates how good it is to be in a particular state s_t , whereas the advantage function $A(S)$ describes the advantage of taking action a_t in state s_t compared to other possible actions. In Equation (32), the final expression for the optimal Q-function can be defined as:

$$Q(s_t, a_t, \theta) = V_{\pi_\theta}(s_t, \theta) + A_{\pi_\theta}(s_t, a_t, \theta) - \frac{1}{|A|} \sum_{a_{t+1}} A(s_t, a_{t+1}, \theta) \quad (32)$$

where $V(S) = V_{\pi_\theta}(s_t, \theta)$ and $A(S) = A_{\pi_\theta}(s_t, a_t, \theta)$. Incorporating the D3QN algorithm's state value function and advantage function into evaluating input states guarantees that these states are objectively assessed. High-dimensional state spaces complicate feature extraction and representation in ultra-low altitude UAV path planning. It exacerbates the issue of Q-value overestimation in traditional Q-networks, which significantly affects policy stability. To address this dual challenge, the present study adopts the D3QN algorithm as an improved approach. The algorithm substantially reduces the risk of Q-value overestimation by effectively integrating a double Q-network structure. Simultaneously, its distinctive dueling architecture decouples the state value function from the advantage function, enhancing the model's capability to capture complex, high-dimensional state features. During algorithm validation, a comparative experimental design was implemented to eliminate the uncer-

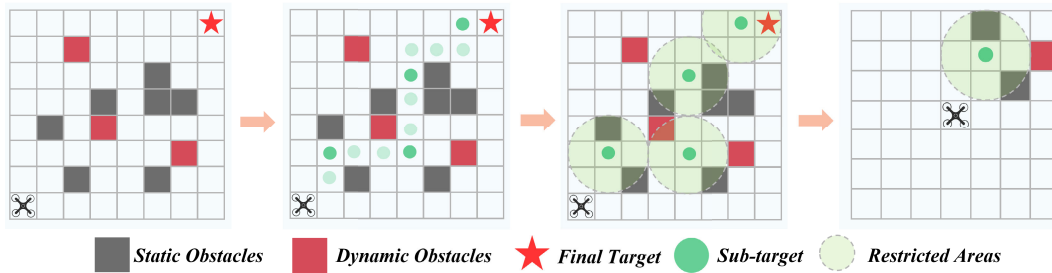


FIGURE 7. The sub-target constraint process.

TABLE 1. Adaptive modifications of greedy strategies.

Time	Value		Difference		I-O
	O-greedy(O)	I-greedy(I)	O-greedy(O)	I-greedy(I)	
0	0.99	0.99	0	0	0
2500	0.86485	0.89773	-0.12515	-0.09227	0.03288
5000	0.7397	0.79962	-0.12515	-0.09811	0.05992
7500	0.6146	0.69269	-0.12515	-0.10693	0.07809
10000	0.48945	0.57616	-0.12515	-0.11653	0.08671
12500	0.36435	0.44917	-0.12515	-0.12699	0.08482
15000	0.2392	0.31076	-0.12515	-0.13841	0.07156
17500	0.1141	0.15993	-0.12515	-0.15083	0.04583
20000	0.01	0.01	-0.12515	-0.14993	0

tainties induced by Q-value overestimation, thus ensuring the reliability and effectiveness of the experimental results.

B. A* ALGORITHM

The A* heuristic graph search method guarantees globally optimal results by expanding search nodes [45], [46] and is widely applied in path planning, particularly within static environments. In this study, A* is employed on static layer maps to estimate paths and enhance UAV trajectory planning for collision-free navigation. The algorithm maintains two dynamically updated lists, OPEN and CLOSE, following these steps: Step 1: Initialization. Insert the initial and target nodes into the algorithm, placing the initial node in the OPEN list where all accessible nodes are recorded. Step 2: Node Selection. From the OPEN list, choose the node with the lowest cost. The cost is defined as:

$$f(n) = h(n) + g(n) \quad (33)$$

where n represents the n -th node, $g(n)$ is the actual distance from the start and $h(n)$ is the predicted distance to the target (computed as per Equation (15)). Step 3: Goal Check. If the chosen node is the target, the search terminates, indicating a path from the start to the target. Otherwise, proceed to node expansion. Step 4: Node Expansion: Expand the neighbors of the chosen node by calculating their costs. Guidelines include: updating a node's cost if a lower one is found in the OPEN list, disregarding nodes already in the CLOSE list (indicating they have been previously searched), and adding new neighbors to the OPEN list. This iterative process

ensures that the optimal path is identified through continuous refinement. It demonstrates that while the A* method is effective in static scenarios, its limitations are evident in more complex contexts.

C. ST-D3QN

The proposed ST-D3QN framework is detailed below. The unique ST-D3QN technique augments the A* algorithm with the DRL algorithm D3QN to drive D3QN training using sub-targets, as shown in Figure 6. Sensors aboard the UAV collect static data about its surroundings and operating conditions. A* computes local static information and estimates an optimum path to the target location using UAV static map data at a given time. The sampled trajectory points are stored sequentially after sampling the projected trajectory with a constant radiation radius. D3QN network training uses sampled path points as sub-targets. The UAV must approach sub-targets. Sub-target restriction generation is shown in Figure 7. The A* algorithm's real-time sub-targets and UAV status data feed the D3QN learning. The grid map provides positional information to the network for learning after secondary processing. Two fully connected layers extract features from network data. Subsequently, the primary network branches into two streams: $V(S)$ and $A(S)$, which respectively estimate the state value $V_{\pi_\theta}(s_t, \theta)$ and action value $A_{\pi_\theta}(s_t, a_t, \theta)$. Finally, the value of the state-action pair $Q(s_t, a_t, \theta)$ is computed according to Equation (32), where the target network outputs the corresponding value of the state-action pair $Q(s_t, a_t, \theta^-)$. Following the network framework of ST-D3QN, the network parameters θ can be defined as:

$$\theta = (\theta_s, \theta_a) \quad (34)$$

where θ_s and θ_a respectively denote the network parameters of $Q(S)$ and $A(A)$. Similarly, the target network parameters θ^- can be represented as follows:

$$\theta^- = (\theta_s^-, \theta_a^-) \quad (35)$$

The network updating mechanism unifies action selection and value estimation, addressing the issue of overestimation. According to equation (28), the loss function $L(\theta)$ of the

Algorithm 1 ST-D3QN (Sub-Target D3QN)

```

1: Initialize all global variables;
2: Initialize the Q network  $Q(s_t, a_t, \theta)$  with random network parameters  $\theta$ ;
3: Initialize A* parameters;
4: Copy the same parameters to the target Q networks  $Q^{\text{target}}(s_t, a_t, \theta^-)$  in D3QN:  $\theta^- \leftarrow \theta$ ;
5: Initialize experience replay buffer  $U$  with size  $U$ ;
6: for each episode in episodes do
7:   Initialize the initial UAV state  $s_0$ ;
8:   for each step in steps do
9:     A* computes the localized sub-target;
10:    Sub-target guide training and constrain rewards;
11:    D3QN agent uses  $\epsilon$ -greedy policy to choose action  $a_t$  in the current state  $s_t$ :

$$a_t = \begin{cases} \text{random } a \in A & \epsilon \\ \arg \max_a Q(s_t, a, \theta) & 1 - \epsilon \end{cases}$$

12:    Execute action  $a_t$  to obtain instant reward  $r_t$  and next state  $s_{t+1}$ ;
13:    The current environment state becomes  $s_{t+1}$ ;
14:    Store  $(s_t, a_t, r_t, s_{t+1})$  in the replay memory  $U$ ;
15:    for each round  $t = 1 \rightarrow T$  do do
16:      Sample one tuple  $(s_t, a_t, r_t, s_{t+1})$  from  $U$ ;
17:      Compute with Q network:

$$Q(s_t, a_t, \theta) = V_{\pi_\theta}(s_t, \theta) + A_{\pi_\theta}(s_t, a_t, \theta) - \frac{1}{|A|} \sum_{a_{t+1}} A(s_t, a_{t+1}, \theta);$$

18:      Update  $\theta$ , minimize the loss function:

$$L(\theta) = E_{\pi_\theta} \left[ (Q(s_t, a_t, \theta^-) - Q(s_t, a_t, \theta))^2 \right];$$

19:      Compute the current network parameters  $\theta$  at time  $t$ :

$$\theta_{t+1} = \theta_t + \gamma \left( Q(s_t, a_t, \theta^-) - Q(s_t, a_t, \theta_t) \right) \nabla_{\theta_t} Q(s_t, a_t, \theta_t);$$

20:      if  $t \bmod T = 0$  then
21:        Update  $\theta^- \leftarrow \theta$ ;
22:      end if
23:    end for
24:  end for
25: end for

```

ST-D3QN network can be defined as:

$$L(\theta) = E_{\pi_\theta} \left[(Q(s_t, a_t, \theta^-) - Q(s_t, a_t, \theta))^2 \right] \quad (36)$$

In Equation (29), we proceed with the gradient update on Equation (34). Therefore, the update of the current network

TABLE 2. ST-D3QN hyperparameters.

Hyperparameters	Value
FC1	256
FC2	256
FC3	128
FC3 for advantage	9
FC3 for value	1
Max episodes	1000 times
Max steps	1000(step)
Max size	1000000
Map size	11 × 11
γ	0.001
$\mathfrak{R}$	20(m)
Mini-batch	64
μ	0.98
network update rate ω	0.001
$\epsilon_{\min}$	0.01

parameters θ can be defined as:

$$\theta_{t+1} = \theta_t + \gamma \left(Q(s_t, a_t, \theta^-) - Q(s_t, a_t, \theta_t) \right) \nabla_{\theta_t} Q(s_t, a_t, \theta_t) \quad (37)$$

According to Equation (31), the parameters θ of the current network are copied to the parameters θ^- of the target network. Figure 8 provides a detailed illustration of the training process of the proposed ST-D3QN algorithm. In the early stages of training, the UAV continuously explores the environment to gather environmental information, which is then stored in the experience replay pool. During the exploration phase, the UAV employs an adaptive ϵ -greedy strategy to select actions. Thus, the actions a_t outputted by the UAV at time t in the early stages can be defined as:

$$a_t = \begin{cases} \text{random } a \in A & \epsilon \\ \arg \max_a Q(s_t, a, \theta) & 1 - \epsilon \end{cases} \quad (38)$$

where the output of actions depends on the greedy factor ϵ , a dynamically set parameter with values ranging between 0 and 1, traditional greedy strategies [47] linearly decrease the greedy factor ϵ to a minimum value without considering the actual application context of UAVs.

A new strategy is suggested for controlling the variation of the greedy factor ϵ differently from traditional methods in this paper. A cutting-edge greedy strategy is proposed tailored explicitly to the intelligent characteristics of UAVs. During the initial learning phase of UAVs, this strategy inclines towards proactively selecting random actions to mitigate the uncertainty stemming from insufficient experience. As UAVs

accumulate learning experience, rapid and autonomous learning becomes achievable, with action selection determined by the network's assessment of the most deterministic action, i.e., an action that yields a maximum Q-value. Thus, at time t , the greedy factor ϵ_t can be defined as:

$$\epsilon_t = \begin{cases} \epsilon_0 + 1 - e^{-\frac{\ln(1+\epsilon_0) * \text{step}}{\eta}} & \epsilon_t \geq 0.01 \\ 0.01 & \epsilon_t < 0.01 \end{cases} \quad (39)$$

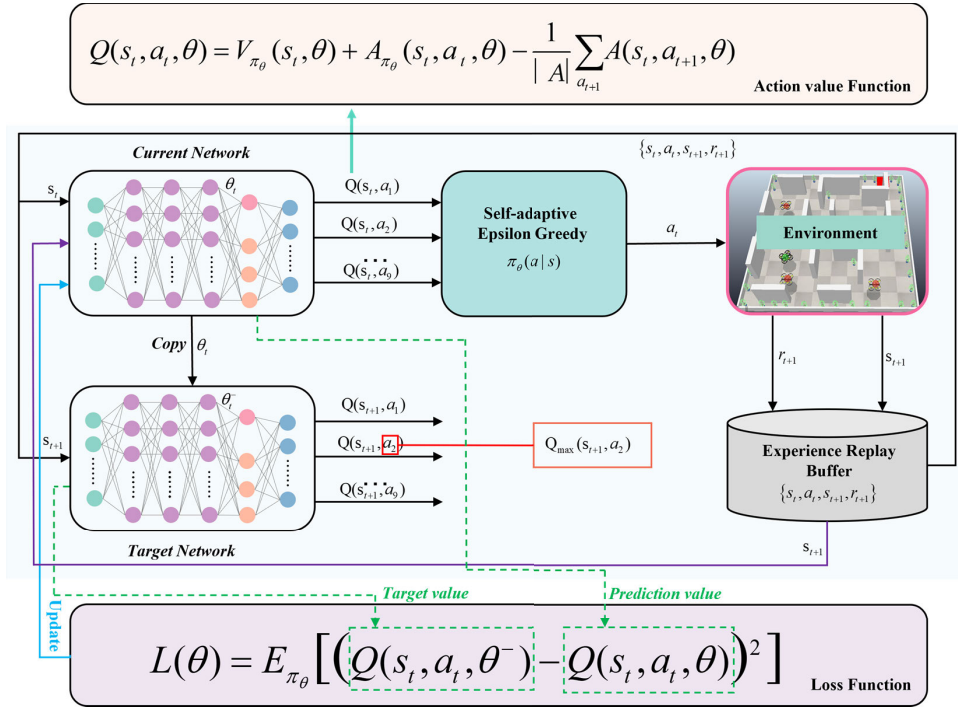


FIGURE 8. The training process of ST-D3QN.

where *step* represents the total learning steps of the UAV during the training process, ε_0 denotes the initial value of the greedy factor, and η is a constant with $\eta = 2.0 \times 10^4$ in this context. ε_t represents the parameter value of the greedy factor ε at time t . ε_0 starts at 0.99 in ST-D3QN and decreases non-linearly throughout training until it reaches the minimal value. Table 1 shows the trend of the modified greedy factor (*I*-greedy), which better meets UAV action selection strategy requirements than the classic greedy factor (*O*-greedy). The modified greedy strategy approach compensates for UAV cognitive learning and experience. Without environmental data, UAVs lack experience and must work hard to learn about their surroundings. UAV expertise grows with experience, allowing them to use Q-values for decision-making. In the early stages, encouraging UAVs to act randomly may help them explore the state space, get feedback, and understand the surroundings.

The improved greedy strategy balances UAV intelligence exploration with exploitation. This invention enhances UAV learning and judgment in ultra-low-altitude settings. For each ST-D3QN network update, tiny batches of experiences are taken from the experience replay pool to train network parameters after preliminary investigation. Equation (32) computes Q-values in the present network, and Equation (36) calculates the loss using the Q-values and goal network values. Equation (37) updates network parameters using gradient descent. Equation (31) copies current network weights to target network weights at a set frequency. Algorithm 1 summarizes ST-D3QN.

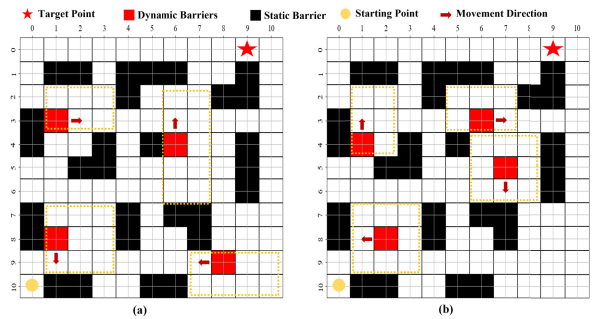


FIGURE 9. Grid-based mapping.

V. EXPERIMENT RESULTS AND ANALYSIS

This study employs the CoppeliaSim simulation platform to conduct simulation experiments. Each method is tested in 50 experiments to evaluate the effectiveness of the proposed ST-D3QN algorithm in UAV path planning for obstacle avoidance. The results demonstrate that the proposed algorithm ensures the efficient and secure completion of UAV missions, highlighting its applicability and feasibility.

The simulation and experimental results of the ST-D3QN algorithm were detailed. The ST-D3QN algorithm efficiently and securely navigates pathways in intricate ultra-low altitude settings. Hence, this work uses total flight time (steps) and distance (length) as performance measurements. Steps refer to the frequency with which the UAV performs operations at predetermined intervals according to the policy. The state transition function dictates its subsequent position. Time and interactions are shown in stages. This paper uses grid

TABLE 3. Performance comparison between the proposed algorithm and other algorithms.

Time	ST-D3QN		D3QN		DDQN		DQN	
	L	P	L	P	L	P	L	P
1	14.31	19.24	15.73	11.23	16.31	7.96	16.9	4.63
2	14.31	25.51	16.1	16.19	15.23	20.72	16.9	12.02
3	15.23	14.05	15.56	12.19	17.22	2.82	17.72	0.00
4	13.77	28.32	17.38	9.53	16.10	16.19	15.31	15.10
5	14.31	7.56	15.21	1.74	15.56	-0.52	17.22	-11.24
6	15.23	14.05	16.31	7.96	16.9	4.63	17.72	0.00
7	14.31	21.97	15.23	16.96	15.73	14.23	16.1	12.21
8	14.31	16.85	15.73	8.60	15.22	11.56	16.9	1.80
9	15.23	25.67	16.90	17.52	19.21	6.25	17.22	15.96
10	14.31	19.24	15.48	12.64	16.31	7.96	16.9	4.63

TABLE 4. Performance comparison between the proposed algorithm and other algorithms(Continued).

Time	ST-D3QN		SAC		IRRT		A*	
	L	P	L	P	L	P	L	P
1	14.31	19.24	15.48	12.64	15.48	12.64	17.72	-
2	14.31	25.51	15.23	20.72	16.9	12.02	19.21	-
3	15.23	14.05	17.21	2.88	15.73	11.23	17.72	-
4	13.77	28.32	14.81	22.90	18.34	4.53	19.21	-
5	14.31	7.56	16.9	-9.17	17.72	-14.47	15.48	-
6	15.23	14.05	15.48	12.64	16.1	9.14	17.72	-
7	14.31	21.97	17.72	3.38	15.48	15.59	18.34	-
8	14.31	16.85	19.24	-11.80	17.22	-0.06	17.21	-
9	15.23	25.67	15.73	23.23	16.31	20.40	20.49	-
10	14.31	19.24	16.10	9.14	16.48	7.00	17.72	-

coordinates. Hence, length may be calculated using the Euclidean formula. The CoppeliaSim UAV environmental simulation platform was initially introduced. Figure 1 illustrates the platform's simulation area of 110 by 110 meters. This simulation employs a radar- and camera-equipped replica UAV to survey the region. The camera sensor is used only for photographic capturing to streamline the experiment. The radar sensor, which has a detection range of 20 meters, is used only for training data. The setting features stationary and non-cooperative UAVs as dynamic obstacles. UAVs construct an 11-by-11 grid map of the simulated world. Figure 9 illustrates a grid map including a yellow circle marking denoting the UAV's initial position and a red star indicating its navigational objective. The image has 12 black grid squares representing real-world impediments such as structures and terrain. Four red grid cells signify dynamic obstacles (UAVs) that emulate diverse, dynamic entities. The dynamic barriers use the same action selection process seen in Figure 3 to function randomly inside their designated activity zones. A yellow dashed border denotes their random assignment on the map. Figures 9(a) and 9(b) illustrate two maps derived from the dynamic obstacle movement zone constraints established in this study. The dynamic obstructions in Figure 9(a) exhibit more dispersion than those in Figure 9(b) at the map's center. The ST-D3QN method is based on Ubuntu20 and NVIDIA GeForce GTX 1050 of 8GB, implemented in Python, using PyTorch. Lua scripts provide autonomous control of UAVs inside the simulation environment.

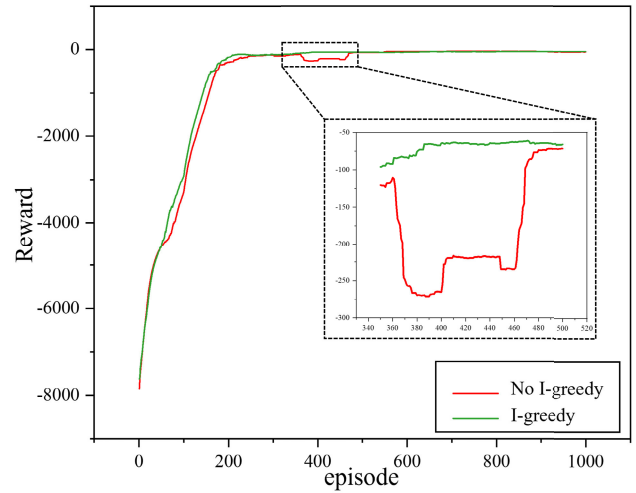

FIGURE 10. The impact of improved greedy strategies.

Table 2 displays the network hyperparameters of the ST-D3QN algorithm, outlining both network and training parameters. The fully connected layers of the network are optimized to handle a large amount of data, with the first layer containing 256 units, the second with 256 units, and the third with 128 units [42]. The fully connected layers responsible for the advantage function contain nine units, while the action function layers contain 1 unit. This paper employs a maximum epoch of 1000 for all training algorithms and sets the UAVs' maximum exploration steps to 1000 for continuous learning and algorithm feasibility assessment. The UAVs' learning rate is fixed at 0.001 [7]. To facilitate efficient training and enhance the learning rate of the ST-D3QN algorithm, we set the minimum batch learning size to 64 and the discount factor to 0.98 [44]. The proposed algorithm network updates occur at a 0.001 update rate between the current and target networks. Furthermore, to allow the algorithm to learn crucial information from the environment, we set the initial and minimum values of the greedy factor in the greedy policy. Expressly, we set the minimum greedy policy factor $\epsilon_{\min}$ is set to 0.01 [44].

Firstly, the impact of improving the greedy strategy on the algorithm is investigated. To independently assess the efficacy of the enhanced greedy technique while mitigating the influence of other optimization variables in ST-D3QN, we chose the fundamental and commonly utilized D3QN as the evaluation algorithm. Figure 10 illustrates that the experimental outcomes reveal comparable performance of both strategies throughout the beginning phase (the first 53 rounds). Beginning with round 53, the adaptive greedy technique enhanced reward increase by dynamically modifying the exploration probability. During the locally magnified time between rounds 340 and 500, the conventional method demonstrated stagnation, while the adaptive greedy strategy transcended the local optimum, resulting in a continuous increase in rewards. These data combined indicate that the adaptive greedy technique markedly improves convergence

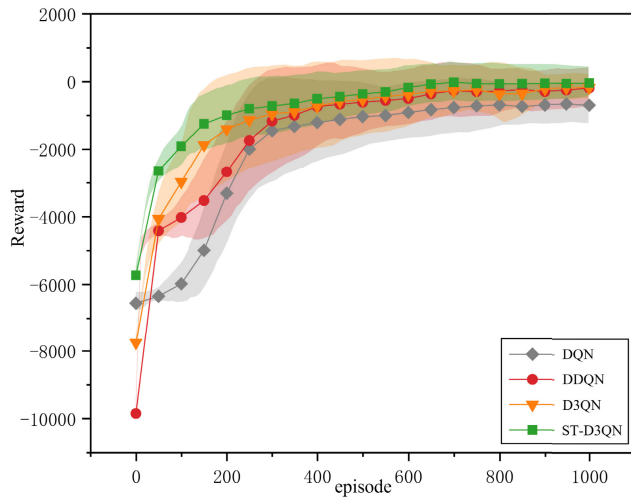


FIGURE 11. Comparison with DQN, DDQN, D3QN.

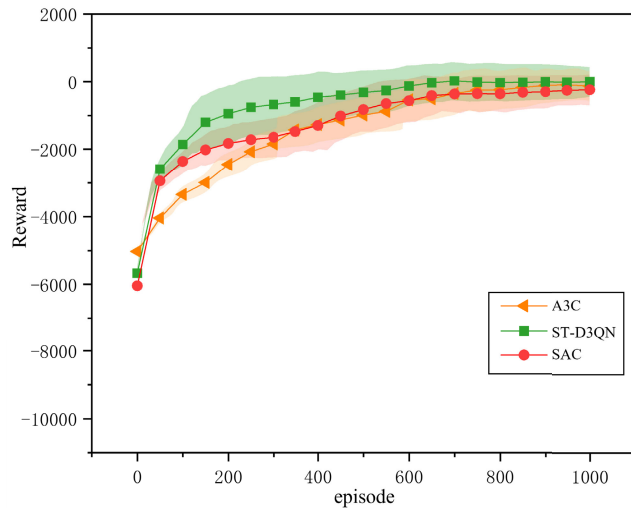


FIGURE 12. Comparison with SAC, A3C.

speed and partially reduces the likelihood of the algorithm becoming ensnared in local optima. Therefore, the improvement of the greedy strategy is effective.

The ST-D3QN algorithm inherits characteristics from the DRL algorithms and other traditional non-RL algorithms. Consequently, two control experiments were conducted during the experimental design process. One set targeted DRL algorithms, while the other targeted traditional non-RL algorithms. Further subdivisions were made into two smaller groups for the experiments concerning DRL algorithms. One group encompassed derivative DRL algorithms of DQN (improved DQN [48], DDQN [49], D3QN [50]), which are continual improvements upon the original DQN algorithm. The other group comprised DRL algorithms with different structures (SAC [51], A3C [25]).

Figure 11 illustrates the comparative results of training rewards between ST-D3QN and other state-of-the-art DRL algorithms, namely DQN, DDQN, and D3QN. Each

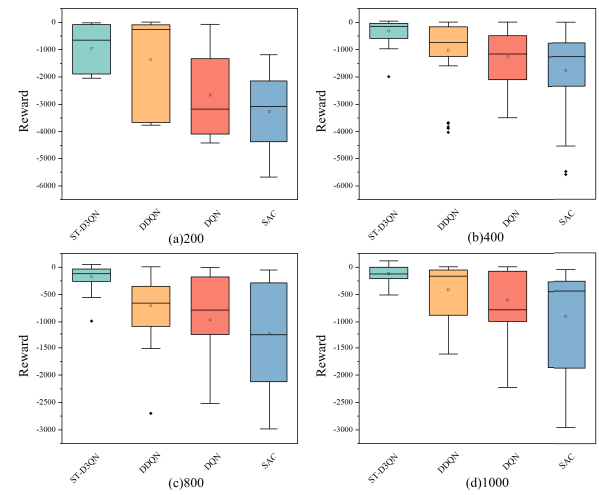


FIGURE 13. Box plots depicting (a) 50 experimental data points from the 200th iteration, (b) 50 experimental data points from the 400th iteration, (c) 50 experimental data points from the 800th iteration, and (d) 50 experimental data points from the 1000th iteration.

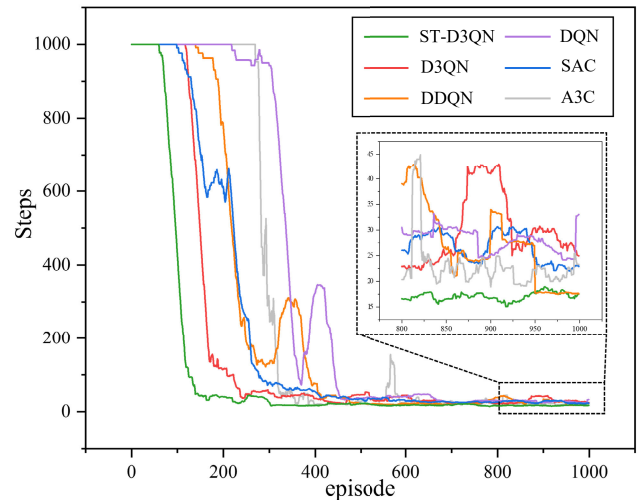


FIGURE 14. Comparison of convergence steps with other deep reinforcement learning algorithms.

algorithm underwent 50 experimental training runs, and the results were averaged. Solid lines represent the average training results of multiple experiments, while the shaded regions indicate the fluctuation range of the data from multiple experiments. DQN algorithm exhibits the least desirable performance, with the most extensive fluctuation range. This is primarily attributed to the issue of overestimation and frequent entrapment in local optima within the training environment.

Additionally, the inadequate reward design of the DQN algorithm leads to sparse rewards, resulting in the slowest convergence speed and significant fluctuations. In contrast, both DDQN and D3QN algorithms, due to the absence of sub-target constraints in their reward mechanisms, face an abundance of dynamic obstacles during training, resulting in a scarcity of rewards for the UAV. To address this issue, the ST-D3QN algorithm introduces the A* algorithm to

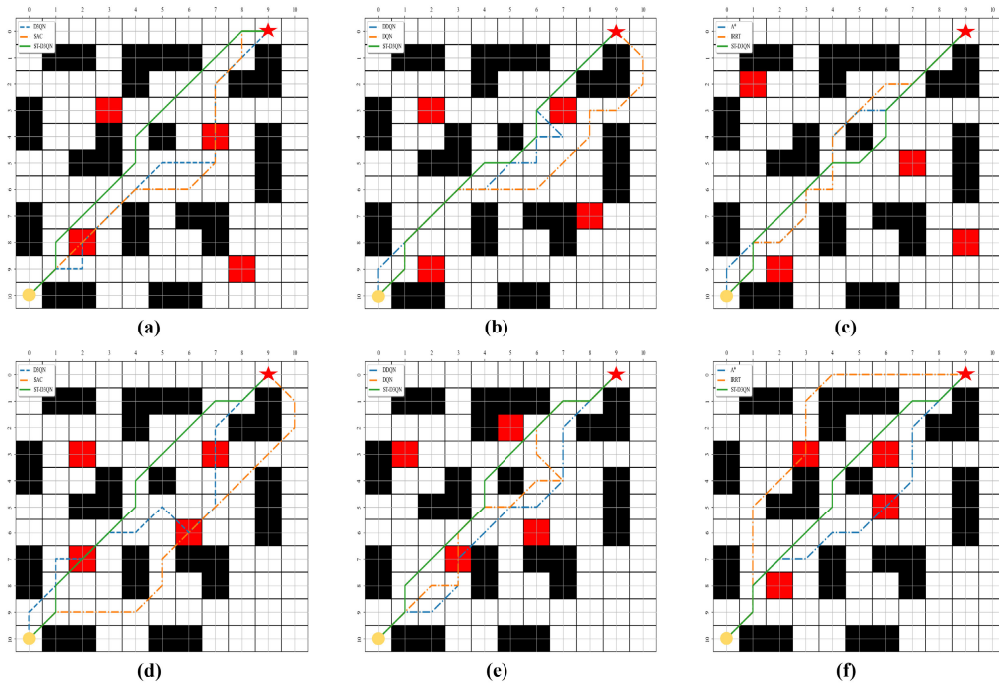


FIGURE 15. Path planning perspective compared with other algorithms.

impose sub-target constraints on rewards, thereby achieving dynamic constraint rewards. The results demonstrate that the ST-D3QN algorithm outperforms other DRL algorithms, with the smallest fluctuation range throughout the training process.

Figure 12 compares the training rewards achieved by the ST-D3QN algorithm against those obtained by other DRL algorithms, such as SAC and A3C. The results indicate that the SAC algorithm exhibits significant limitations in exploring unfamiliar regions due to its low sample efficiency, high reward sensitivity, and the absence of a sub-target constraint mechanism. Although entropy regularization provides certain benefits, it is insufficient to address the continuously evolving policy demands of dynamic environments, resulting in a convergence speed that is markedly slower than that of ST-D3QN. In contrast, while the A3C algorithm demonstrates relatively stable training rewards—as evidenced by minor fluctuations—it falls short in convergence speed during path planning experiments, mainly because of its lack of explicit sub-target constraints and reliance on synchronous updates. Overall, the experimental findings robustly validate the significant advantages of the ST-D3QN algorithm in terms of both convergence speed and adaptability in dynamic environments.

The data extracted from Figures 11 and 12 are the training results (average reward convergence) of all algorithms at the 200th, 400th, 800th, and 1000th rounds. These results are transformed into box plots, as illustrated in Figure 13. It is evident from these plots that the experimental data of the ST-D3QN algorithm exhibit relatively high density and

concentration compared to other algorithms. This observation indicates the stability of the ST-D3QN algorithm.

Figure 14 depicts the convergence stages of ST-D3QN alongside various DRL algorithms. Sparse rewards lead the SAC algorithm to get ensnared in local optima, yielding steady but inconsistent development. The A3C method, devoid of a sub-target restriction, exhibits unstructured learning characterized by delayed convergence and increased unpredictability in step counts after convergence. Likewise, constrained early-stage rewards result in variations in DDQN and DQN between episodes 300 and 500. The lack of an adaptive greedy approach and sub-target direction in D3QN and sparse rewards leads to network instability and oscillatory behavior post-convergence. Conversely, ST-D3QN exhibits much quicker convergence and enhanced stability, decreasing the time UAV allocates to environmental path planning.

After undergoing 50 experimental trials alongside other UAV path planning algorithms, the ST-D3QN algorithm proved to be the top performer in minimizing path length (L) and relative percentage (P). The relative percentage value, P , quantifies the performance of path planning algorithms compared to the A* algorithm. The value P reflects the relative change in path length compared to the A* algorithm. If $P > 0$, the algorithm outperforms the A* algorithm. Conversely, if $P < 0$, the algorithm underperforms compared to the A* algorithm. It is explicitly delineated as presented in Equation (40):

$$P = \frac{L_{A^*} - L_{al}}{L_{A^*}} \times 100\% \quad (40)$$

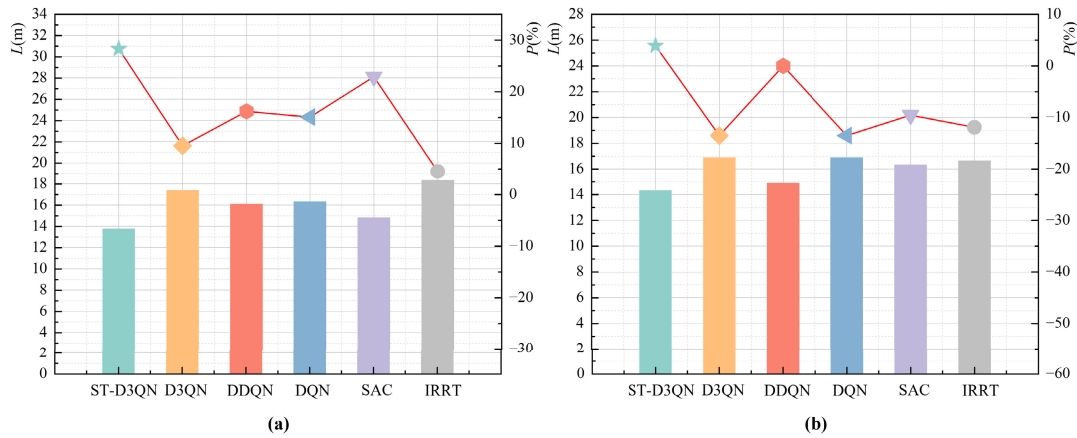


FIGURE 16. Comparison of statistical significance and relative percentage of the shortest path.

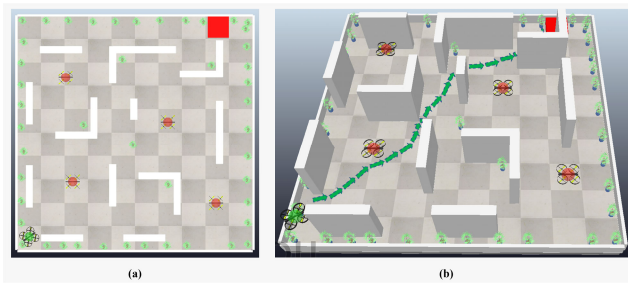


FIGURE 17. Environment validation.

where L_{A^*} represents the path length planned by the A* algorithm. L_{al} represents the path length planned by other algorithms. Tables 3 and 4 demonstrate that the ST-D3QN algorithm yields the shortest pathways in comparison to other approaches, as seen in Figure 15. Figure 16 conducts a statistical analysis of these findings using data from two sets of experimental maps (Figures 15(a)–15(c) for Figure 16(a) and Figures 15(d)–15(f) for Figure 16(b)). The findings indicate that ST-D3QN provides shorter and more stable pathways while competing algorithms exhibit more fluctuation in P-values. Furthermore, Figure 16 illustrates the algorithm's efficient real-time UAV path planning inside a simulated environment, whereby the UAV successfully arrives at its destination without collisions.

The computational complexity [44] analysis of ST-D3QN on edge devices compares value-based methods (DQN, DDQN, D3QN) with policy gradient approaches (SAC, A3C). DQN's complexity is mainly driven by neural network forward and backward passes, scaling with the number of parameters. Although DDQN and D3QN add constant overhead via target networks and dueling architectures, their complexity remains similar. SAC incurs slightly higher computational costs due to simultaneous updates of the policy and two Q-networks with probabilistic sampling and entropy regularization. In contrast, A3C's asynchronous multi-process updates increase costs through extra communication and synchronization. Furthermore,

despite IRRT's potential worst-case exponential complexity, adequate sampling limits its search space, and the A* algorithm benefits from a heuristic design that reduces grid nodes. Consequently, ST-D3QN alternates between D3QN and A* to segment global path planning into sub-targets, thus decreasing the reinforcement learning state space and maintaining a minimal computational load suitable for edge devices.

In summary, in comparison to other six path planning algorithms, the proposed ST-D3QN algorithm can optimize paths (shortest paths) and real-time obstacle avoidance in ultra-low-altitude environments, thereby reducing the airborne flight time of UAVs (minimizing the number of steps) and enhancing the efficiency of UAV mission execution. The experiment results indicate that the ST-D3QN algorithm outperforms other algorithms in terms of speed, stability, and convergence.

VI. CONCLUSION

This paper introduces the ST-D3QN, an innovative algorithm that optimizes and augments the capabilities of the D3QN and A* algorithms for sub-target guidance. This optimization aims to address the complexities of navigation planning for UAVs in ultra-low-altitude settings. The proposed ST-D3QN algorithm also optimizes the sparsity of rewards in deep reinforcement learning environments. To rigorously evaluate the performance of the ST-D3QN algorithm, a series of comparative assessments were conducted against other state-of-the-art algorithms on simulation platforms. The experimental results underscore the viability and superiority of the ST-D3QN algorithm over other classical DRL algorithms. Furthermore, it outperforms traditional path-planning algorithms such as IRRT and A*. The ST-D3QN algorithm has proven effective in training UAVs to identify the shortest path, ensuring safe navigation, and completing path-planning tasks within complex environments with multiple UAVs operating without inter-UAV communication. Furthermore, the approach intermittently amalgamates D3QN with the

A* algorithm by partitioning the comprehensive planning process into more tractable local sub-targets. This method substantially decreases overall processing complexity, rendering its implementation on edge devices viable.

The ST-D3QN algorithm developed in this paper has demonstrated significant efficacy in two-dimensional path planning for a single uncrewed aerial vehicle, and its foundational principles suggest substantial potential for adaptation to three-dimensional planning contexts. The feasibility of three-dimensional path planning is achieved by including z data with x and y coordinates. Presently, studies are conducted on a fixed-altitude assumption, a constraint that may restrict its usefulness in scenarios with considerable height changes. Simultaneously, the technique is also suitable for multi-UAV cooperative tasks, provided UAVs operate independently without communication interference. Future research will address real-world difficulties related to multi-dimensional dynamic changes and inter-agent collaboration to improve the flexibility and resilience of the algorithm in complex environments.

REFERENCES

- [1] L. Chu, Q. Li, F. Gu, X. Du, Y. He, and Y. Deng, "Design, modeling, and control of morphing aircraft: A review," *Chin. J. Aeronaut.*, vol. 35, no. 5, pp. 220–246, May 2022.
- [2] F. Nex, C. Armenakis, M. Cramer, D. A. Cucci, M. Gerke, E. Honkavaara, A. Kukko, C. Persello, and J. Skaloud, "UAV in the advent of the twenties: Where we stand and what is next," *ISPRS J. Photogramm. Remote Sens.*, vol. 184, pp. 215–242, Feb. 2022.
- [3] A. Heidari, N. Jafari Navimipour, M. Unal, and G. Zhang, "Machine learning applications in Internet-of-Drones: Systematic review, recent deployments, and open issues," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–45, Dec. 2023.
- [4] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, "Path planning techniques for mobile robots: Review and prospect," *Expert Syst. Appl.*, vol. 227, Oct. 2023, Art. no. 120254.
- [5] X. Wang, L. T. Yang, D. Meng, M. Dong, K. Ota, and H. Wang, "Multi-UAV cooperative localization for marine targets based on weighted subspace fitting in SAGIN environment," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5708–5718, Apr. 2022.
- [6] J. Fu, G. Sun, J. Liu, W. Yao, and L. Wu, "On hierarchical multi-UAV Dubins traveling salesman problem paths in a complex obstacle environment," *IEEE Trans. Cybern.*, vol. 54, no. 1, pp. 123–135, Jan. 2023.
- [7] Z. Zhang, S. Wang, J. Chen, and Y. Han, "A bionic dynamic path planning algorithm of the micro UAV based on the fusion of deep neural network optimization/filtering and hawk-eye vision," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 6, pp. 3728–3740, Jun. 2023.
- [8] V. P. Tran, A. Perera, M. A. Garratt, K. Kasmarik, and S. G. Anavatti, "Coverage path planning with budget constraints for multiple unmanned ground vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12506–12522, Nov. 2023.
- [9] M. Lyu, Y. Zhao, C. Huang, and H. Huang, "Unmanned aerial vehicles for search and rescue: A survey," *Remote Sens.*, vol. 15, no. 13, p. 3266, Jun. 2023.
- [10] J. Fan, X. Chen, and X. Liang, "UAV trajectory planning based on bi-directional APF-RRT algorithm with goal-biased," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 119137.
- [11] Y. Tan, W. Yi, P. Chen, and Y. Zou, "An adaptive crack inspection method for building surface based on BIM, UAV and edge computing," *Autom. Construct.*, vol. 157, Jan. 2024, Art. no. 105161.
- [12] C.-W. Fu, M.-L. Ku, Y.-J. Chen, and T. Q. S. Quek, "UAV trajectory, user association, and power control for multi-UAV-enabled energy-harvesting communications: Offline design and online reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9781–9800, Mar. 2024.
- [13] Y. Xue and W. Chen, "Multi-agent deep reinforcement learning for UAVs navigation in unknown complex environment," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 2290–2303, Jan. 2024.
- [14] H. Huang and A. V. Savkin, "Path planning for a solar-powered UAV inspecting mountain sites for safety and rescue," *Energies*, vol. 14, no. 7, p. 1968, Apr. 2021.
- [15] Y. Guo, X. Liu, Q. Jia, X. Liu, and W. Zhang, "HPO-RRT: A sampling-based algorithm for UAV real-time path planning in a dynamic environment," *Complex Intell. Syst.*, vol. 9, no. 6, pp. 7133–7153, Dec. 2023.
- [16] Z. Pan, C. Zhang, Y. Xia, H. Xiong, and X. Shao, "An improved artificial potential field method for path planning and formation control of the multi-UAV systems," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1129–1133, Mar. 2022.
- [17] J. Rao, C. Xiang, J. Xi, J. Chen, J. Lei, W. Giernacki, and M. Liu, "Path planning for dual UAVs cooperative suspension transport based on artificial potential field—A algorithm," *Knowl.-Based Syst.*, vol. 277, Oct. 2023, Art. no. 110797.
- [18] J. Gao, Y. Zheng, K. Ni, Q. Mei, B. Hao, and L. Zheng, "Fast path planning for firefighting UAV based on A-star algorithm," *J. Phys., Conf. Ser.*, vol. 2029, no. 1, Sep. 2021, Art. no. 012103.
- [19] N. Bolourian and A. Hammad, "LiDAR-equipped UAV path planning considering potential locations of defects for bridge inspection," *Autom. Construct.*, vol. 117, Sep. 2020, Art. no. 103250.
- [20] Z. Zhang, J. Jiang, J. Wu, and X. Zhu, "Efficient and optimal penetration path planning for stealth unmanned aerial vehicle using minimal radar cross-section tactics and modified A-Star algorithm," *ISA Trans.*, vol. 134, pp. 42–57, Mar. 2023.
- [21] J. Ou, X. Guo, M. Zhu, and W. Lou, "Autonomous quadrotor obstacle avoidance based on dueling double deep recurrent Q-learning with monocular vision," *Neurocomputing*, vol. 441, pp. 300–310, Jun. 2021.
- [22] F. Kong, Q. Wang, S. Gao, and H. Yu, "B-APFDQN: A UAV path planning algorithm based on deep Q-network and artificial potential field," *IEEE Access*, vol. 11, pp. 44051–44064, 2023.
- [23] Y. Peng, Y. Liu, D. Li, and H. Zhang, "Deep reinforcement learning based freshness-aware path planning for UAV-assisted edge computing networks with device mobility," *Remote Sens.*, vol. 14, no. 16, p. 4016, Aug. 2022.
- [24] Y. Zhou, J. Shu, X. Zheng, H. Hao, and H. Song, "Real-time route planning of unmanned aerial vehicles based on improved soft actor-critic algorithm," *Frontiers Neurobot.*, vol. 16, Dec. 2022, Art. no. 1025817.
- [25] X. Zhu, T. Qiu, W. Qu, X. Zhou, Y. Wang, and D. O. Wu, "Path planning for adaptive CSI map construction with A3C in dynamic environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2925–2937, May 2023.
- [26] M. Yan, C. Aun Chan, A. F. Gyga, C. Li, A. Nirmalathas, and I. Chih-Lin, "Efficient generation of optimal UAV trajectories with uncertain obstacle avoidance in MEC networks," *IEEE Internet Things J.*, vol. 11, no. 23, pp. 38380–38392, Dec. 2024.
- [27] M. Yan, R. Xiong, Y. Wang, and C. Li, "Edge computing task offloading optimization for a UAV-assisted Internet of Vehicles via deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5647–5658, Apr. 2024.
- [28] W. Guan and K. Wang, "Autonomous collision avoidance of unmanned surface vehicles based on improved A-Star and dynamic window approach algorithms," *IEEE Intell. Transp. Syst. Mag.*, vol. 15, no. 3, pp. 36–50, May 2023.
- [29] L. Kästner, X. Zhao, Z. Shen, and J. Lambrecht, "A hybrid hierarchical navigation architecture for highly dynamic environments using time-space optimization," in *Proc. IEEE/SICE Int. Symp. Syst. Integr. (SII)*, Jan. 2023, pp. 1–8.
- [30] X. Wang, M. C. Gursay, T. Erpek, and Y. E. Sagduyu, "Learning-based UAV path planning for data collection with integrated collision avoidance," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16663–16676, Sep. 2022.
- [31] M. Kim, J. Kim, M. Jung, and H. Oh, "Towards monocular vision-based autonomous flight through deep reinforcement learning," *Expert Syst. Appl.*, vol. 198, Jul. 2022, Art. no. 116742.
- [32] X. Wang and M. C. Gursay, "Resilient path planning for UAVs in data collection under adversarial attacks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2766–2779, 2023.
- [33] K. Yu, H. Li, L. Xing, T. Wen, D. Fu, Y. Yang, C. Zhou, R. Chang, S. Zhao, L. Xing, and H. Bai, "Scene-aware refinement network for unsupervised monocular depth estimation in ultra-low altitude oblique photography of UAV," *ISPRS J. Photogramm. Remote Sens.*, vol. 205, pp. 284–300, Nov. 2023.

- [34] M. Dai, E. Zheng, Z. Feng, L. Qi, J. Zhuang, and W. Yang, "Vision-based UAV self-positioning in low-altitude urban environments," *IEEE Trans. Image Process.*, vol. 33, pp. 493–508, 2024.
- [35] R. She and Y. Ouyang, "Efficiency of UAV-based last-mile delivery under congestion in low-altitude air," *Transp. Res. C, Emerg. Technol.*, vol. 122, Jan. 2021, Art. no. 102878.
- [36] T. Wang, W. Du, C. Jiang, Y. Li, and H. Zhang, "Safety constrained trajectory optimization for completion time minimization for UAV communications," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34482–34491, Nov. 2024.
- [37] J. Liu, F. Yang, X. Wang, L. Qu, M. Jin, and H. Dai, "Joint optimization of charging station placement and UAV trajectory for fresh data collection," *IEEE Internet Things J.*, vol. 11, no. 14, pp. 25057–25073, Jul. 2024.
- [38] R. Zhang, M. Wang, L. X. Cai, and X. Shen, "Learning to be proactive: Self-regulation of UAV based networks with UAV and user dynamics," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4406–4419, Jul. 2021.
- [39] B. Khamidehi and E. S. Sousa, "Reinforcement-Learning-Aided safe planning for aerial robots to collect data in dynamic environments," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13901–13912, Aug. 2022.
- [40] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992.
- [41] A. J. Mersereau, P. Rusmevichientong, and J. N. Tsitsiklis, "A structured multiarmed bandit problem and the greedy policy," *IEEE Trans. Autom. Control*, vol. 54, no. 12, pp. 2787–2802, Dec. 2009.
- [42] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [43] W. Huang, C. Zhang, J. Wu, X. He, J. Zhang, and C. Lv, "Sampling efficient deep reinforcement learning through preference-guided stochastic exploration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 12, pp. 18553–18564, Dec. 2024.
- [44] C. Yan, C. Wang, X. Xiang, Z. Lan, and Y. Jiang, "Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing UAVs using local situation maps," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1260–1270, Feb. 2022.
- [45] K. I. Trovato and L. Dorst, "Differential A," *IEEE Trans. Knowl. Data Eng.*, vol. 14, no. 6, pp. 1218–1229, Jun. 2002.
- [46] L. Pasandi, M. Hooshmand, and M. Rahbar, "Modified A algorithm integrated with ant colony optimization for multi-objective route-finding; Case study: Yazd," *Appl. Soft Comput.*, vol. 113, Dec. 2021, Art. no. 107877.
- [47] M. Zhang, S. Wu, J. Jiao, N. Zhang, and Q. Zhang, "Energy- and cost-efficient transmission strategy for UAV trajectory tracking control: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8958–8970, May 2023.
- [48] J. Fan, X. Zhang, and Y. Zou, "Hierarchical path planner for unknown space exploration using reinforcement learning-based intelligent frontier selection," *Expert Syst. Appl.*, vol. 230, Nov. 2023, Art. no. 120630.
- [49] Z. Chu, F. Wang, T. Lei, and C. Luo, "Path planning based on deep reinforcement learning for autonomous underwater vehicles under ocean current disturbance," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 1, pp. 108–120, Jan. 2023.
- [50] X. Wang and M. C. Gurfey, "Resilient UAV path planning for data collection under adversarial attacks," in *Proc. ICC-IEEE Int. Conf. Commun.*, May 2022, pp. 625–630.
- [51] P. Chen, J. Pei, W. Lu, and M. Li, "A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance," *Neurocomputing*, vol. 497, pp. 64–75, Aug. 2022.



ANPING YANG was born in Guizhou, China, in 1997. He received the B.S. degree in communications engineering from Jiangsu University of Science and Technology, Zhenjiang, China, where he is currently pursuing the M.S. degree in computer technology. His research interests include machine learning and reinforcement learning.



JING HUAN received the Ph.D. degree from Naval Aviation University, Yantai, China, in 2013. She is currently a Senior Lecturer with the School of Computer Science and Engineering, Jiangsu University of Science and Technology, Zhenjiang, China. Her research interests include computer vision and artificial intelligence.



QI WANG (Member, IEEE) received the B.S. degree from Shandong University of Finance and Economics, Jinan, China, in 2005, the M.S. degree from Sungkyunkwan University, Suwon, South Korea, in 2011, and the Ph.D. degree from the University of Trento, Trento, Italy, in 2015. He was a Visiting Scholar with North Carolina State University, Raleigh, USA, from 2013 to 2014. He is currently a Lecturer with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China. He has authored or co-authored several top journal articles, he is also a reviewer for a couple of journals and conferences. His research interests include wireless communication, smart cities, and the Internet of Things.



HUALONG YU (Member, IEEE) was born in Harbin, in 1982. He received the B.S. degree in computer science from Heilongjiang University, Harbin, in 2005, and the M.S. and Ph.D. degrees in computer science from Harbin Engineering University, Harbin, China, in 2008 and 2010, respectively. Since 2020, he has been a Professor with the School of Computer Science and Engineering, Jiangsu University of Science and Technology, Zhenjiang, China. From 2013 to 2017, he was a Postdoctoral Fellow with the School of Automation, Southeast University, Nanjing, China. From 2017 to 2018, he was a Senior Visiting Scholar with the Faculty of Information Technology, Monash University, Melbourne, Australia. He has authored or co-authored more than 90 journals and conference papers and four monographs. His research interests include machine learning, data mining, and bioinformatics. He is a member in the organizing committee of several international conferences. He is also a member of ACM, China Computer Federation (CCF), and the Youth Committee of the Chinese Association of Automation (CAA). He is an active Reviewer for more than 20 high quality international journals, including IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON CYBERNETICS, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON COMPUTATIONAL BIOLOGY AND BIOINFORMATICS, and ACM TKDD. He is an Associate Editor of IEEE ACCESS.



SHANG GAO (Member, IEEE) received the B.S. degree in system engineering from Air Force Engineering University, Xi'an, China, in 1993, the M.S. degree in military equipment from Air Force Engineering University, Xi'an, in 1996, and the Ph.D. degree in pattern recognition and intelligent systems from Nanjing University of Science and Technology, Nanjing, China, in 2006. Since 2009, he has been a Professor with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China, where he has been the Dean of the School of Computer, since 2017. He has published over 80 research articles on the professional journals and conferences. His research interests include optimization theory, swarm intelligence, and machine learning.

...